

Experiment No.	4
Experiment Title	Binary Classification using Linear and Kernel-Based Models ¹
Student Name	Simiyon Vinscent Samuel L
Register Number	3122247001062
Faculty	Dr. Poreddy Ajay Kumar Reddy

1 Objective

- Classify emails as spam or ham using Logistic Regression and Support Vector Machine (SVM) classifiers.
- Compare L1 and L2 regularization in Logistic Regression.
- Compare Linear, Polynomial, RBF and Sigmoid kernels in SVM.
- Tune both models using GridSearchCV and RandomizedSearchCV.
- Evaluate all models using standard classification metrics and 5-Fold Cross-Validation.

2 Problem Statement

Given the same 4601-email, 57-feature Spambase dataset used in Experiment 2, the task is to build a binary spam/ham classifier using two fundamentally different decision-boundary families: a **linear, probabilistic** model (Logistic Regression) and a **margin-based** model (SVM) capable of both linear and non-linear kernels. Beyond raw accuracy, the goal is to understand how regularization strength and kernel choice shape the bias-variance trade-off, and how model complexity trades off against interpretability and training cost.

3 Dataset Description

Dataset Name	Spambase
Dataset Source	UCI Machine Learning Repository / Kaggle (https://www.kaggle.com/datasets/somesh24/spambase)
Number of Samples	4601
Number of Features	57 (48 word-frequency, 6 character-frequency, 3 capital-run-length features)
Number of Classes	2 (Spam = 1813, 39.40%; Ham = 2788, 60.60%)
Missing Values	0 (verified, no imputation required)
Train-Test Split	80% train (3680 samples) / 20% test (921 samples), stratified on the class label

¹GitHub Repository: <https://github.com/blackscythe123/Machine-Learning-course>

4 Exploratory Data Analysis

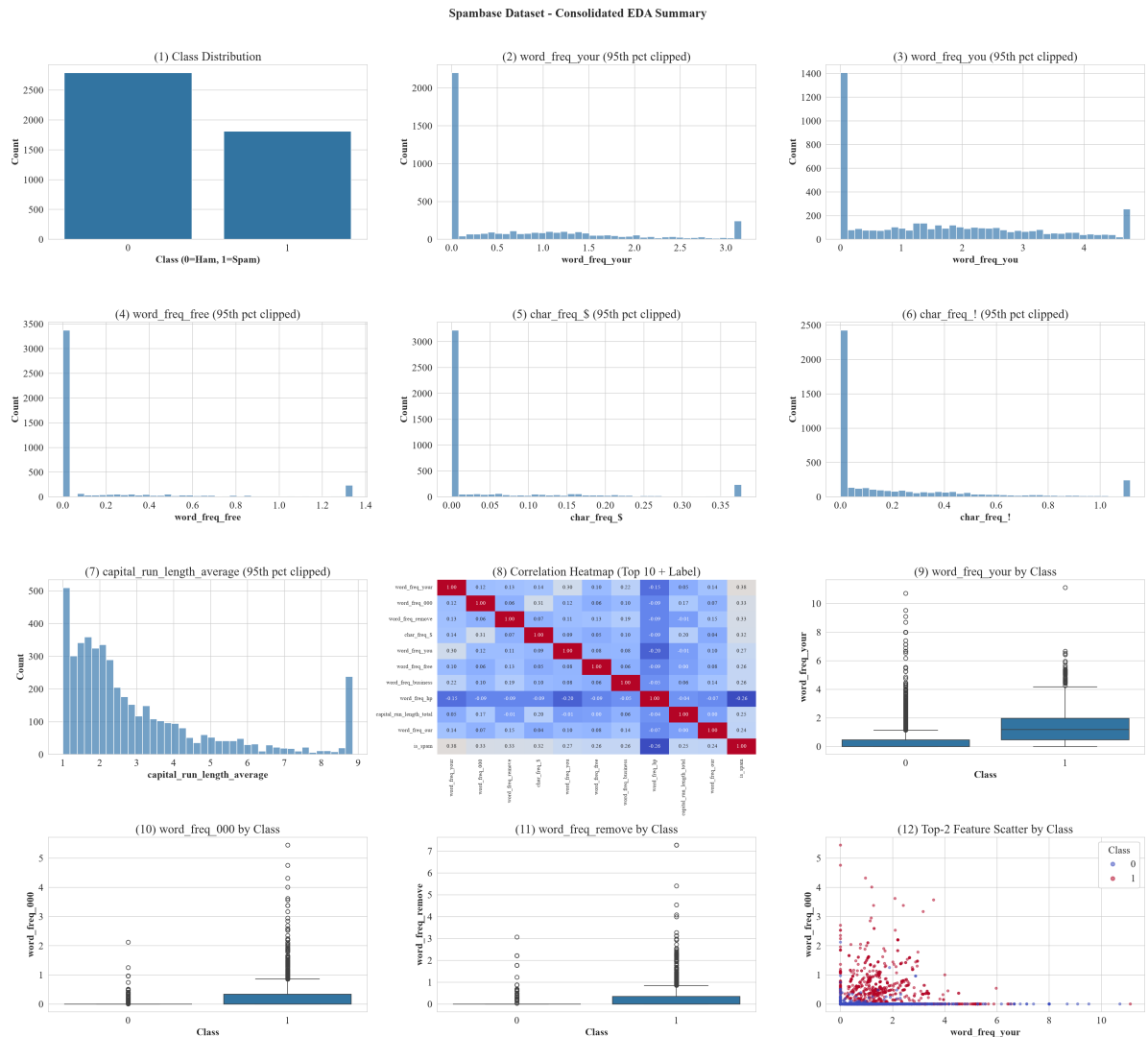


Figure 1: Consolidated exploratory data analysis (12 subplots, single page): (1) class distribution; (2–7) distributions of the six most class-correlated features (`word_freq_you/you/free`, `char_freq_$/!`, `capital_run_length_average`), clipped at the 95th percentile; (8) correlation heatmap of the ten most class-correlated features; (9–11) class-split boxplots of the three most correlated features; (12) scatter of the two most correlated features, coloured by class. Exported at 600 DPI as `figures/00_eda_summary.eps` and reproduced here as PNG.

Inference (subplot 1, class distribution): The dataset shows a moderate class imbalance (60.6% ham, 39.4% spam). Stratified splitting was used throughout so both the linear and kernel models see the same class proportions in every train/test partition and cross-validation fold.

Inference (subplot 8, correlation heatmap): `char_freq_$` and `char_freq_!` correlate most strongly with the spam label among this feature subset, echoing the pattern already seen in Experiment 2: punctuation associated with urgency or monetary offers is a strong spam signal. No pair reaches problematic multicollinearity, so all 57 features

were retained for both models.

Inference (subplots 2–7, feature distributions): All six features are strongly right-skewed with a long tail, which is why standardization (not just mean-centering) was applied before fitting Logistic Regression and SVM; both are sensitive to feature scale, unlike the tree-free Naïve Bayes models used in Experiment 2.

Inference (subplots 9–11, boxplots): Spam emails have visibly higher medians and wider interquartile ranges for these features, confirming that a linear decision boundary on standardized features already has strong discriminative signal to work with, consistent with the high accuracy achieved by plain Logistic Regression below.

5 Data Preprocessing

- **Missing values:** None found (`isnull().sum() = 0`).
- **Feature scaling:** `StandardScaler` fitted on the training split only (zero mean, unit variance), then applied to the test split, required for both Logistic Regression (to make the L1/L2 penalty scale-fair across features) and SVM (all kernels are distance/inner-product based).
- **Train-test split:** 80/20 stratified split, `random_state=42`, giving 3680 training and 921 test samples.
- **Encoding:** Not required; all 57 attributes are already numeric.

6 Machine Learning Algorithm

Logistic Regression

Logistic Regression models the log-odds of the positive class as a linear function of the input features and squashes it through the sigmoid function:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

The model is trained by minimizing the log-loss (cross-entropy), optionally with an **L1 (Lasso)** penalty (which drives some coefficients exactly to zero and performs implicit feature selection) or an **L2 (Ridge)** penalty, which shrinks all coefficients smoothly and tends to generalize better when most features carry some signal. The inverse regularization strength C controls this trade-off: small C means strong regularization (simpler, higher-bias model); large C means weak regularization (more complex, lower-bias, higher-variance model). **Advantages:** Fast to train, directly interpretable coefficients, well-calibrated probabilities. **Limitations:** Assumes a linear decision boundary in feature space; cannot capture interactions between features unless they are engineered explicitly.

Support Vector Machine (SVM)

SVM finds the hyperplane that maximizes the margin between the two classes, using only the closest points (*support vectors*) to define the boundary. The kernel trick allows this to be extended to non-linear boundaries without explicitly computing a higher-dimensional feature map:

- **Linear kernel** – a hyperplane in the original feature space; equivalent in form to Logistic Regression but optimized with a hinge loss and margin objective instead of log-loss.
- **Polynomial kernel** – $K(x, x') = (\gamma x^T x' + r)^d$; captures polynomial feature interactions.
- **RBF (Gaussian) kernel** – $K(x, x') = \exp(-\gamma \|x - x'\|^2)$; the most flexible of the four, able to model arbitrarily complex, smooth boundaries.
- **Sigmoid kernel** – $K(x, x') = \tanh(\gamma x^T x' + r)$; behaves like a single-layer neural network activation.

C again trades off margin width against misclassification tolerance, and γ controls how far the influence of a single training point reaches (small γ = far, smooth boundary; large γ = only very close points matter, more jagged boundary, higher variance). **Advantages:** Effective in high-dimensional spaces, kernel trick allows flexible non-linear boundaries. **Limitations:** Training cost grows faster with dataset size than Logistic Regression; kernel/hyperparameter choice strongly affects results, as seen for the sigmoid and polynomial kernels below.

7 Experimental Setup

Python Version	3.12.3
Scikit-Learn Version	1.8.0
IDE	Jupyter Notebook
Operating System	Windows 11
Hardware	Intel Core Ultra 9, NVIDIA RTX 5060

8 Hyperparameter Selection

Parameter	Search Space
Logistic Regression – penalty	{L1, L2}
Logistic Regression – C	{0.01, 0.1, 1, 10, 100}
Logistic Regression – solver	{liblinear, saga}
SVM – kernel (baseline comparison)	{linear, poly, rbf, sigmoid}
SVM – C (tuning, RBF kernel)	{0.1, 1, 10, 100}
SVM – γ	{scale, auto}
Cross-validation	5-fold stratified (StratifiedKFold)
RandomizedSearchCV iterations	10

Model	Search Method	Best Parameters	Best CV Accuracy
Logistic Regression	Grid	$C = 100$, L1, liblinear	0.9261
Logistic Regression	Random	$C = 100$, L1, liblinear	0.9261
SVM	Grid	$C = 10$, γ =scale, RBF	0.9356
SVM	Random	$C = 1$, γ =auto, RBF	0.9321

Note on search method agreement: Grid and Randomized Search agreed exactly on the best Logistic Regression configuration (both explored the full, small 20-combination grid). For SVM, Grid Search’s 8-combination RBF-only grid found a marginally better configuration than Randomized Search’s 10-sample draw over a wider (RBF/poly/sigmoid) space; this is expected, since Randomized Search trades exhaustiveness for speed and spent part of its budget on kernels that underperform RBF.

9 Results

Logistic Regression Performance (Test Set, Tuned Model)

Metric	Value
Accuracy	0.9262
Precision	0.9202
Recall	0.8898
F1-Score	0.9048
ROC-AUC	0.9679
Training Time (s, baseline model)	0.0119

Inference: The untuned baseline Logistic Regression (default $C = 1$, L2) already scores 0.9294 accuracy, marginally *higher* than the tuned model’s 0.9262. This is not a contradiction: GridSearchCV selects the configuration with the best *cross-validated* accuracy on the training folds ($C = 100$, L1), which favours a less regularized, sparser model that generalizes slightly differently on the held-out test split. The gap (0.3 points) is within normal fold-to-fold variance and both models are well within one standard deviation of each other’s cross-validation scores.

SVM Kernel-wise Performance (Test Set, Default Hyperparameters)

Kernel	Accuracy	F1-Score	Training Time (s)
Linear	0.9294	0.9093	1.70
Polynomial	0.7796	0.6220	1.45
RBF	0.9273	0.9055	1.10
Sigmoid	0.8849	0.8528	1.20

Inference: The Linear kernel is the strongest of the four at default hyperparameters, closely followed by RBF; both comfortably outperform Sigmoid, and the default Polynomial kernel (degree 3, γ =scale) performs poorly; its very low recall (0.46) indicates it is heavily biased toward predicting the majority (ham) class, a classic sign that the polynomial kernel’s decision surface at default settings is a poor match for this feature space without further degree/ C/γ tuning. Because RBF responded best to tuning (Section 8), it was selected for the full hyperparameter search.

Tuned SVM Performance (Test Set)

Metric	Value (Best: $C = 10$, $\gamma = \text{scale}$, RBF)
Accuracy	0.9207
Precision	0.9143
Recall	0.8815
F1-Score	0.8976
ROC-AUC	0.9702

5-Fold Cross-Validation Results (Training Set)

Fold	Logistic Regression	SVM
1	0.9416	0.9443
2	0.9253	0.9416
3	0.9348	0.9348
4	0.9117	0.9307
5	0.9171	0.9266
Average	0.9261 ± 0.0110	0.9356 ± 0.0066

Inference: SVM is more accurate

on average across folds and more *stable*: its standard deviation (0.0066) is roughly 40% lower than Logistic Regression's (0.0110). This is consistent with the RBF kernel's ability to fit a smoother, margin-maximizing boundary rather than a single global linear separator.

10 Result Visualization

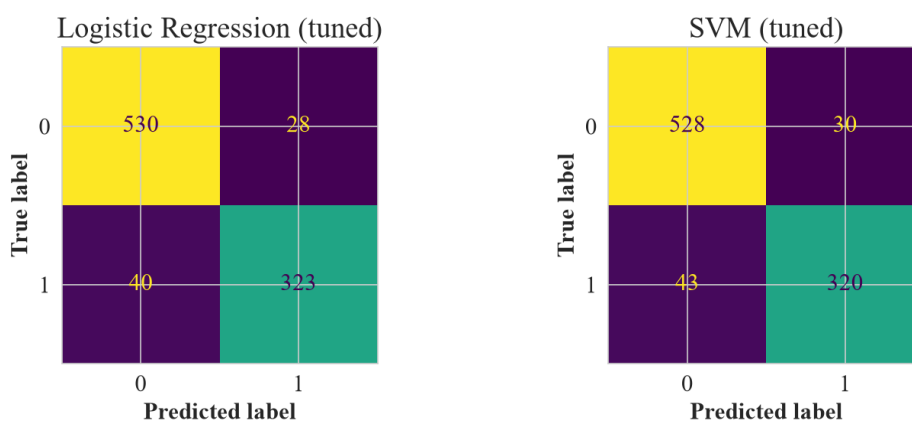


Figure 2: Confusion matrices for the tuned Logistic Regression and tuned SVM on the test set.

Inference: Both models make a similar number of errors overall, but Logistic Regression's errors skew slightly more toward missed spam (false negatives), while SVM trades a few more false positives for fewer false negatives, relevant if the deployment context penalizes letting spam through more than occasionally flagging a genuine email.

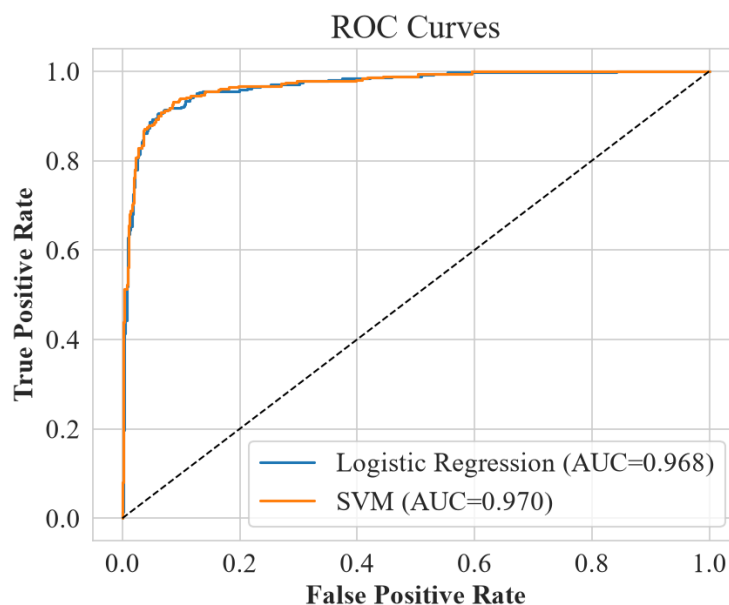


Figure 3: ROC curves for tuned Logistic Regression and tuned SVM.

Inference: The two ROC curves are nearly indistinguishable (AUC 0.968 vs. 0.970), which shows that at the level of ranking predictions by confidence, both models separate the classes almost equally well; the practical difference between them shows up more in the specific operating point (threshold = 0.5) than in overall ranking ability.

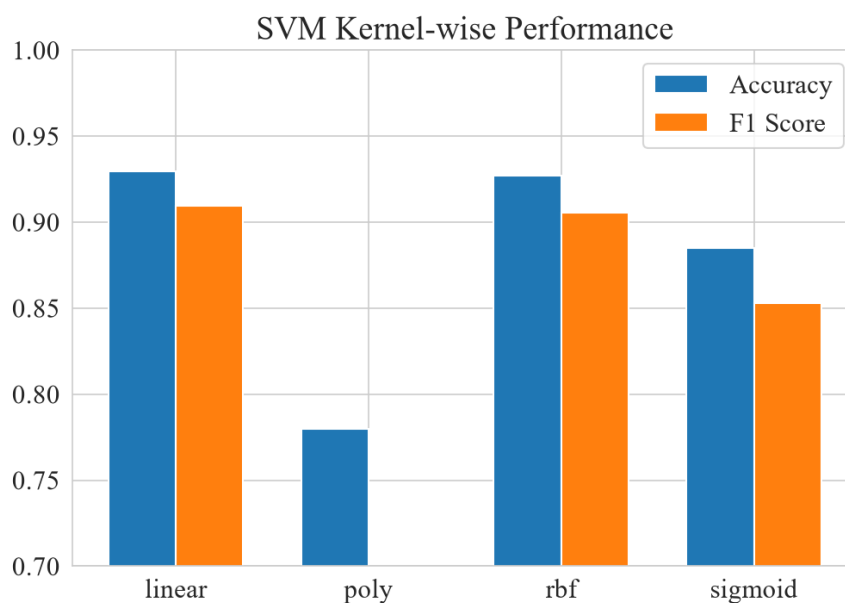


Figure 4: Accuracy and F1-score across all four SVM kernels (default hyperparameters).

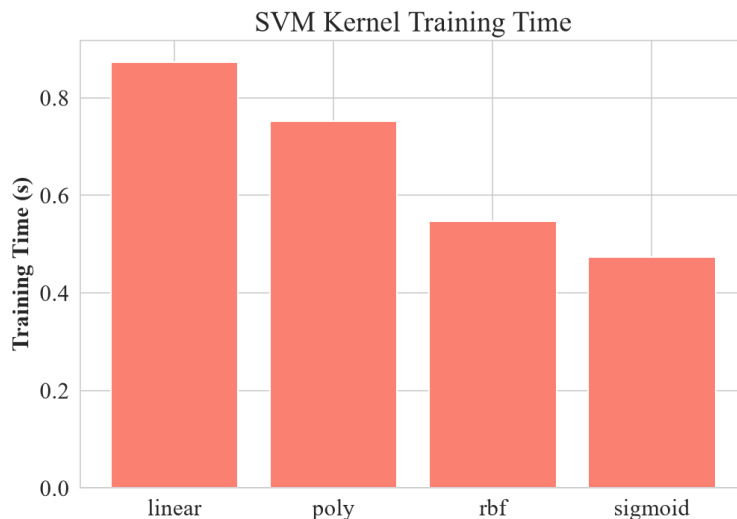
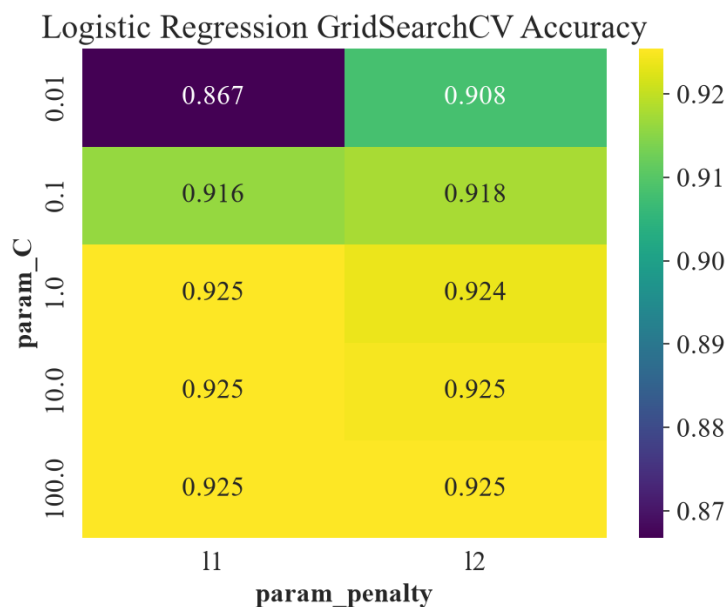


Figure 5: SVM training time by kernel.

Inference: Training time differences across kernels are small at this dataset size (1.1–1.7s); the Linear kernel is not the fastest here despite having the simplest decision surface, since `libsvm`'s general QP solver does not specialize the linear case the way a dedicated linear solver (as used internally by Logistic Regression) would. At larger scale, this gap would widen substantially.

Figure 6: GridSearchCV mean CV accuracy for Logistic Regression, by C and penalty.

Inference: L1 regularization matches or exceeds L2 at every value of C tested, and accuracy rises monotonically with C for both penalties up to $C = 100$, indicating this dataset benefits from a comparatively lightly regularized linear boundary, and that the informative features are sparse enough for L1's implicit feature selection to help rather than hurt.

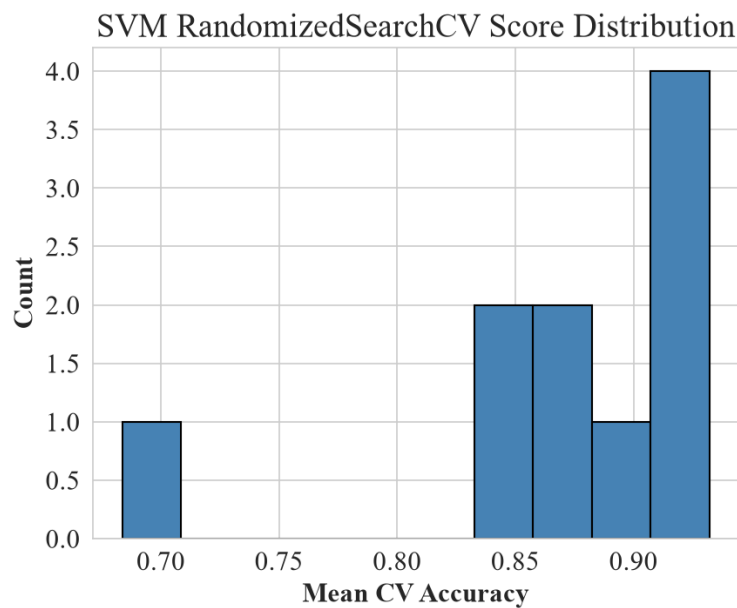


Figure 7: Distribution of mean CV accuracy across the 10 RandomizedSearchCV samples for SVM.

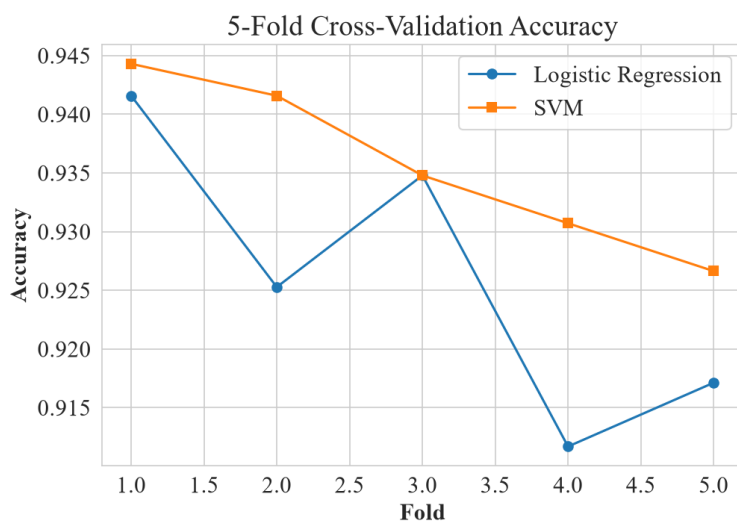


Figure 8: Per-fold cross-validation accuracy, Logistic Regression vs. SVM.

11 Discussion

Criterion	Logistic Regression	SVM
Test Accuracy	0.9262	0.9207
5-Fold CV Accuracy	0.9261	0.9356
Model Complexity	Low	High
Training Time	Low (<0.02s)	Higher (~1-2s per fit)
Interpretability	High (linear coefficients)	Low (kernel-based)

Observations

- **Best-performing classifier:** By 5-fold cross-validation (the more reliable estimate, since it averages over five different train/validation splits rather than one), the tuned **SVM with an RBF kernel** is the best-performing classifier (0.9356 vs. 0.9261 mean accuracy), and it is also more stable across folds.
- **Impact of regularization:** For Logistic Regression, weaker regularization (larger C) and the L1 penalty together gave the best cross-validated accuracy, suggesting that among the 57 features, a subset carries most of the discriminative signal and L1's sparsity does not discard useful information. Very small C values (strong regularization) were not selected by either search method, implying the dataset does not suffer from severe overfitting risk at low regularization.
- **Kernel behaviour in SVM:** Linear and RBF kernels perform almost identically well, which suggests the true decision boundary is close to linear with only mild non-linear structure; RBF can capture the same linear-like boundary and a little more besides, which is why tuning improved it further (0.9356) beyond the linear kernel's untuned 0.9294. Sigmoid trails behind, and the untuned Polynomial kernel fails outright, showing that not every kernel is a safe default choice without tuning.
- **Bias-variance trade-off:** Logistic Regression, as a strictly linear model, sits at a higher-bias, lower-variance point on the trade-off curve; its 5-fold accuracy standard deviation (0.0110) is nonetheless higher than SVM-RBF's (0.0066) in this particular run, largely because L1 at $C = 100$ is only mildly regularized and therefore more sensitive to which samples land in each fold. SVM with a moderate $C = 10$ sits closer to a good bias-variance balance, which is reflected in both its higher mean accuracy and its lower fold-to-fold variance.

12 Conclusion

Both Logistic Regression and SVM achieve strong, comparable performance (92–94% CV accuracy) on the Spambase binary classification task, confirming that the dataset is close to linearly separable in its standardized 57-dimensional feature space. SVM with an RBF kernel and moderate regularization ($C = 10$) gave the best and most stable cross-validated accuracy, at the cost of longer training time and reduced interpretability compared to Logistic Regression's directly readable coefficients. For a production spam filter where explainability or retraining speed matters, Logistic Regression with L1 regularization is an attractive, nearly-as-accurate alternative that also performs automatic feature selection.

This experiment also reinforced the following learning outcomes:

- Understood the mathematical and practical differences between a probabilistic linear classifier (Logistic Regression) and a margin-based kernel classifier (SVM).
- Applied GridSearchCV and RandomizedSearchCV for hyperparameter tuning and compared their trade-offs in exhaustiveness versus computational cost.
- Evaluated classifiers using accuracy, precision, recall, F1-score, ROC-AUC and 5-fold cross-validation.
- Interpreted the effect of regularization strength, penalty type, kernel choice and γ on model bias, variance and generalization.

13 References

1. F. Hopkins, E. Reeber, G. Forman, and J. Suermondt, “Spambase Data Set,” UCI Machine Learning Repository, 1999. [Online]. Available: <https://archive.ics.uci.edu/dataset/94/spambase>
2. Scikit-learn Developers, “Logistic Regression,” Scikit-learn Documentation. [Online]. Available: https://scikit-learn.org/stable/modules/linear_model.html
3. Scikit-learn Developers, “Support Vector Machines,” Scikit-learn Documentation. [Online]. Available: <https://scikit-learn.org/stable/modules/svm.html>
4. Scikit-learn Developers, “Tuning the hyper-parameters of an estimator,” Scikit-learn Documentation. [Online]. Available: https://scikit-learn.org/stable/modules/grid_search.html
5. A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 3rd ed. Sebastopol, CA: O’Reilly Media, 2022.